

A Spiking Neural Network tutorial: Your first SNN

André Furlan

Rua Cristóvão Colombo 2265, Jd Nazareth, 15054-000, São José do Rio Preto - SP, Brazil.

Instituto de Biociências, Letras e Ciências Exatas, Unesp - Univ Estadual Paulista (São Paulo State University)

São José do Rio Preto, Brazil

Email: ensismoebius@gmail.com

Abstract—

The growing demand for increased processing power with deeper neural networks results in escalating energy consumption. Spiking Neural Networks (SNNs) emerge as efficient alternatives to conventional neural networks, especially when implemented on neuromorphic hardware, which mimics the operating principles of biological neurons to reduce energy usage. Although frameworks like SNNtorch provide ready-to-use tools, understanding the theoretical foundations and practical implementation of SNNs remains essential, particularly for deployment in resource-constrained environments or on specialized architectures where such frameworks may not be feasible. This tutorial presents the fundamental concepts of spiking neurons and guides the reader through their implementation in C++.

Index Terms—Spiking Neural Networks, SNN, LIF

I. INTRODUCTION

Standard Neural Networks (NNs), as defined here —*multilayered, fully connected, with or without recurrent or convolutional layers*— require that all neurons be activated during both forward and backward passes. This means that every unit in the network must perform computations at every time step, leading to significant power consumption [1]. In an era where responsible use of energy resources is essential, a critical question arises: how can we design NN models that are energy-efficient enough to be deployed on portable or resource-constrained devices?

Spiking Neural Networks (SNNs) offer a compelling answer to this problem. As the third generation of neural networks, SNNs encode information using discrete spike events and update neuron states only when necessary. This event-driven paradigm mimics biological neuronal dynamics and allows for sparse computations, which are particularly advantageous when implemented on neuromorphic hardware. Such systems can achieve high energy efficiency by leveraging the inherent sparsity of spike-based communication.

Although frameworks like SNNtorch provide accessible tools for simulating and training SNNs, understanding their theoretical underpinnings and low-level implementation is crucial—especially in scenarios where existing frameworks are incompatible with specialized hardware or minimal runtime environments.

The objective of this paper is to guide the reader from the theoretical foundations of spiking neurons to the practical implementation of a simple SNN layer in C++, using surrogate gradient descent for supervised learning.

The remaining of this document is organized as follows: Section II presents the fundamentals of SNNs, Section III details the tests and the results and lastly, Section IV is dedicated to the conclusions that are followed by Section V that presents the future works, then the references and an appendix.

II. BIOLOGICAL AND COMPUTATIONAL BACKGROUND

Biological sensory systems convert external stimuli—such as light, odors, touch, and taste—into discrete electrical events known as **spikes**. A spike is a voltage pulse that conveys information through the nervous system [5]. These signals are propagated along neural pathways to be processed, ultimately generating behavioral or physiological responses to the environment.

A biological neuron emits a spike only when the accumulated excitatory input at the soma exceeds a certain threshold. In the absence of such stimulation, the neuron remains inactive. This event-driven behavior makes biological neurons highly efficient in terms of energy consumption and computational resources.

Spiking Neural Networks (SNNs) aim to emulate some of these advantages by processing **spikes** at the input, hidden, and output layers. While SNNs can also accept continuous-valued inputs, they retain their event-driven properties and energy efficiency due to sparse activation patterns.

It is important to note that SNNs are **not** detailed simulations of biological neurons. Instead, they approximate specific computational characteristics inspired by neurobiology. More biologically faithful models—such as those explored in [4], which incorporate dendritic nonlinearities and other advanced neuronal features—have shown remarkable results in classification tasks, but at the cost of greater computational complexity.

As illustrated in Figure 1, SNN neurons exhibit strong noise tolerance when appropriately parameterized, functioning as **low-pass filters**. They can still generate reliable spikes under considerable interference. Moreover, SNN neurons are inherently time-sensitive, making them especially effective for processing temporal or sequential data streams [1].

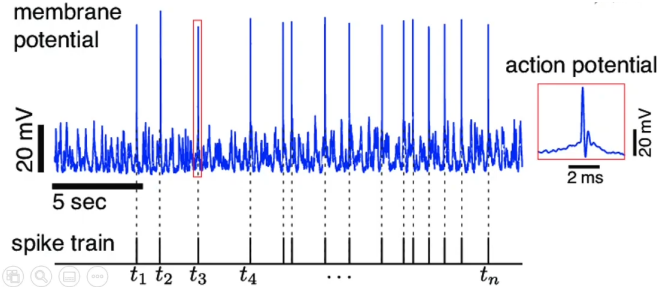


Fig. 1: Spikes from a noisy signal. Source [3]

A. Leaky Integrate and Fire Neuron

Although the focus of this work is the *Leaky Integrate-and-Fire* (LIF) neuron—due to its simplicity, computational efficiency, and its strong generalization performance across many tasks [3]—there exist more biologically accurate models, such as the **Hodgkin–Huxley neuron** [2], and others like the one proposed in [4], which incorporate dendritic nonlinearities and additional biophysical properties to more closely replicate real neuronal behavior.

Unlike a standard artificial neuron, LIF integrates incoming weighted inputs over time with a *leakage* mechanism that continuously reduces the membrane potential, simulating the passive decay observed in biological membranes. Only when the membrane potential crosses a threshold does the neuron emit a spike, after which the potential resets or decreases.

LIF behaves much like Resistor-Capacitor circuit as illustrated in Figure 2. In this analogy R represents the leakage resistance, I_{in} the input current, C the membrane capacitance, U_{mem} the membrane potential (i.e., the accumulated charge) and ϑ a switch that lets the capacitor discharge (i.e. emit a spike) when a voltage threshold is reached.

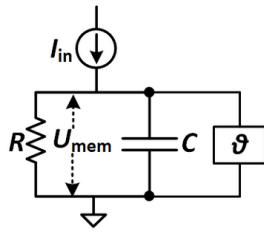


Fig. 2: RC model: Source: [1]

Spikes are represented as **sparsely** distributed **ones** in a train of **zeros**, as illustrated in Figure 3 and Figure 4. This approach simplify the models and reduces the computational power and needed storage to run a SNN.

Spikes, Sparsity and Static-Suppression

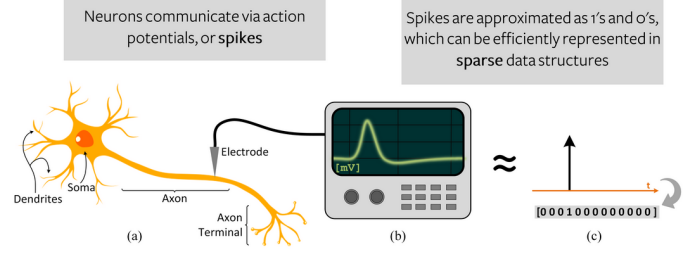


Fig. 3: Sparsity on Spike Neuron Networks. Source: [1]

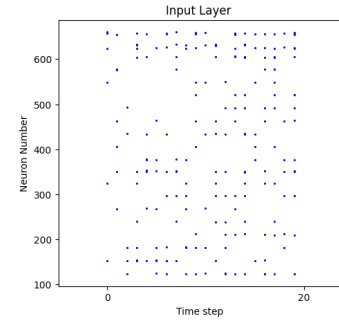


Fig. 4: Sparse activity of a SNN: The horizontal axis represents the time step of some data being processed the vertical are the SNN neuron number. Note that, most of the time, very few neurons gets activated. Source: The author.

As a result of the aforementioned characteristics, information in SNNs is encoded in terms of the *timing* and/or *rate* of spikes. This enables effective processing of temporal data streams but imposes limitations when dealing with static data.

B. Mathematical Model of the LIF Neuron

LIF model is governed by the Equations [1] bellow:

Let's consider Q as a measurement of electrical charge and $V_{mem}(t)$ is the membrane potential in a certain time t than the neuron capacitance C is given by Equation 1.

$$C = \frac{Q}{V_{mem}(t)} \quad (1)$$

Therefore the neuron charge can be expressed as Equation 2.

$$Q = C \cdot V_{mem}(t) \quad (2)$$

To know how these charge changes according to the time (aka current) we can derivate Q as in Equation 3. This expression express the current in the capacitive part of the neuron I_C

$$I_C = \frac{dQ}{dt} = C \cdot \frac{dV_{mem}(t)}{dt} \quad (3)$$

To calculate the total current passing by the resistive part of the circuit we may use the Ohm's law:

$$V_{mem}(t) = R.I_R \implies I_R = \frac{V_{mem}(t)}{R} \quad (4)$$

Then considering that the total current do not change, as seen in Figure 5, we have the total input current I_{in} of the neuron as in Equation 5.

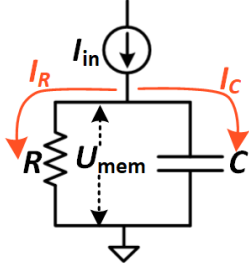


Fig. 5: RC model for currents: $I_{in} = I_R + I_C$

$$I_{in}(t) = I_R + I_C \implies I_{in}(t) = \frac{V_{mem}(t)}{R} + C \cdot \frac{dV_{mem}(t)}{dt} \quad (5)$$

Therefore, to describe the passive membrane we got a linear Equation 6.

$$\begin{aligned} I_{in}(t) &= \frac{V_{mem}(t)}{R} + C \cdot \frac{dV_{mem}(t)}{dt} \implies \\ I_{in}(t) - \frac{V_{mem}(t)}{R} &= C \cdot \frac{dV_{mem}(t)}{dt} \implies \\ \boxed{R \cdot I_{in}(t) - V_{mem}(t) &= R \cdot C \cdot \frac{dV_{mem}(t)}{dt}} \end{aligned} \quad (6)$$

Then, if we consider $\tau = R \cdot C$ as the **membrane time constant** we get voltages on both sides of Equation 7 which describes the RC circuit.

$$\begin{aligned} R \cdot I_{in}(t) - V_{mem}(t) &= R \cdot C \cdot \frac{dV_{mem}(t)}{dt} \implies \\ R \cdot I_{in}(t) - V_{mem}(t) &= \tau \cdot \frac{dV_{mem}(t)}{dt} \implies \\ \boxed{\tau \cdot \frac{dV_{mem}(t)}{dt} &= R \cdot I_{in}(t) - V_{mem}(t)} \end{aligned} \quad (7)$$

From that, and setting $I_{in} = 0$ (i.e. no input) and considering $\tau = R \cdot C$ is a constant and a starting voltage $V_{mem}(0)$ the neuron's voltage behavior can be modeled as an exponential curve as can be seen in Equation 8.

$$\begin{aligned} \tau \cdot \frac{dV_{mem}(t)}{dt} &= R \cdot I_{in}(t) - V_{mem}(t) \implies \\ \tau \cdot \frac{dV_{mem}(t)}{dt} &= -V_{mem}(t) = \\ e^{\ln(V_{mem}(t))} &= e^{-\frac{t}{\tau}} = \\ \boxed{V_{mem}(t) &= V_{mem}(0) \cdot e^{-\frac{t}{\tau}}} \end{aligned} \quad (8)$$

Then one can say that: In the absence of an input I_{in} , the membrane potential decays exponentially as illustrated in Figure 6 and implemented in Listing 1.

```
1 def lif(V_mem, dt=1, I_in=0, R=5, C=1):
2     tau = R*C
3     V_mem = V_mem + (dt/tau)*(-V_mem + I_in*R)
4     return V_mem
```

Listing 1: Python implementation of the action potential decaying of a LIF: $I_{in} = 0$

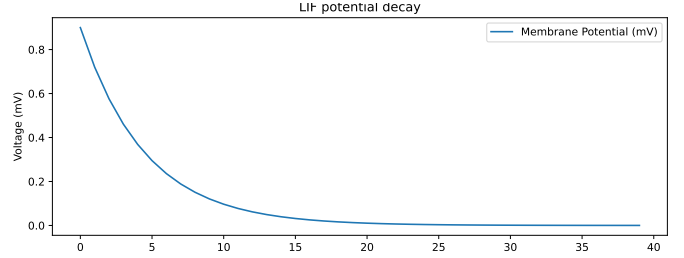


Fig. 6: Membrane potential decaying. Source: The author

With the results from Equation 7 it is possible to calculate the action potential increasing as seen in Equation 9.

$$\begin{aligned} \tau \cdot \frac{dV_{mem}(t)}{dt} &= R \cdot I_{in}(t) - V_{mem}(t) = \\ \frac{dV_{mem}(t)}{dt} + \frac{V_{mem}(t)}{\tau} &= \frac{R \cdot I_{in}(t)}{\tau} \implies \\ \text{Integrating factor: } e^{\int \frac{1}{\tau} dt} &= e^{\frac{1}{\tau} t} \implies \\ (e^{\frac{1}{\tau} t} \cdot V_{mem}(t))' &= \frac{R \cdot I_{in}(t)}{\tau} \cdot e^{\frac{1}{\tau} t} = \\ \int (e^{\frac{1}{\tau} t} \cdot V_{mem}(t))' &= \int \frac{R \cdot I_{in}(t)}{\tau} \cdot e^{\frac{1}{\tau} t} dt = \\ e^{\frac{1}{\tau} t} \cdot V_{mem}(t) &= \int \frac{R \cdot I_{in}(t)}{\tau} \cdot e^{\frac{1}{\tau} t} dt \therefore \\ \text{Considering: } V_{mem}(t=0) &= 0 \implies \\ \boxed{V_{mem}(t) &= I_{in}(t) \cdot R(1 - e^{-\frac{t}{\tau}})} \end{aligned} \quad (9)$$

Note that when action potentials increases there is still an exponential behavior as seen in Figure 7 and implemented in Listing 2.

```
1 def lif(V_mem, dt=1, I_in=1, R=5, C=1):
2     tau = R*C
3     V_mem = V_mem + (dt/tau)*(-V_mem + I_in*R)
4     return V_mem
```

Listing 2: Python implementation of the action potential decreasing of a LIF: $I_{in} = 1$

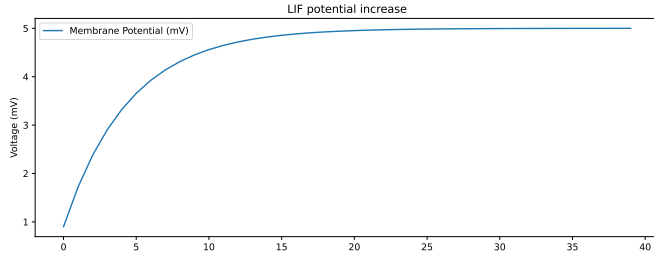


Fig. 7: Membrane potential increasing. Source: The author

Then taking into account some **threshold** which indicates a reset into the neuron potential and two type of resets (to zero and threshold subtraction) finally is possible to model the full LIF behavior as depicted in Figure 8 and implemented in Listing 3:

```
1 def lif(V_mem, dt=1, I_in=1, R=5, C=1, V_thresh = 2,
2       reset_zero = True):
3     tau = R*C
4     V_mem = V_mem + (dt/tau)*(-V_mem + I_in*R)
5     if V_mem > V_thresh:
6         if reset_zero:
7             V_mem = 0
8         else:
9             V_mem = V_mem - V_thresh
10    return V_mem
```

Listing 3: Python implementation of the action potential full simulation of a LIF: $I_{in} = 1$, $V_{thresh} = 2$ is threshold

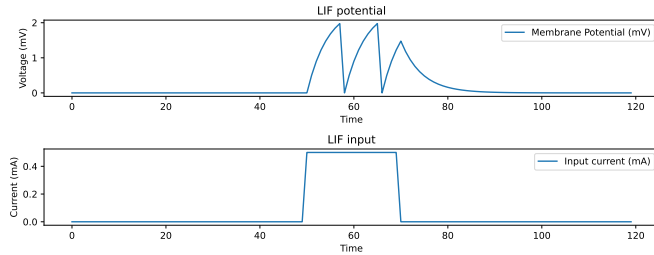


Fig. 8: Plot of the full simulated LIF: A 0.5 mA was provided from time 51 to 70.

C. Training

How do SNNs get trained? Well, this is still an open question. A SNN neuron has an activation-function behavior that is more relatable to a **step function**. Therefore, in principle, we can't use gradient descent-based solutions because this kind of function is **not** differentiable [5].

But there are some insights out there that may shed some light on this subject: While some *in vivo/in vitro* observations show that brains, in general, learn by strengthening/weakening and adding/removing synapses or even by creating new neurons or other cumbersome methods like RNA packets, there are some more acceptable ones like the ones in the list below [5]:

- **Surrogate Gradient Descent:** Approximates the step function by using another mathematical function, which

is differentiable (like a sigmoid), in order to train the network. These approximations are used only in the backward pass, while keeping the step function in the forward pass [5].

- **Spike Timing-Dependent Plasticity (STDP):** If a pre-synaptic neuron fires **before** the post-synaptic one, there is a strengthening in connection, but if the post-synaptic neuron fires before, then there is a weakening.
- **Evolving Algorithms:** Use the selection of the fittest throughout many generations of networks.
- **Reservoir/Dynamic Computing:** Echo state networks or Liquid state machines respectively.

The one used in the SNN made in this work were **Surrogate Gradient Descent**.

III. EXPERIMENTS

1) Neural networks:

A. Full connected spiking neurons SNN

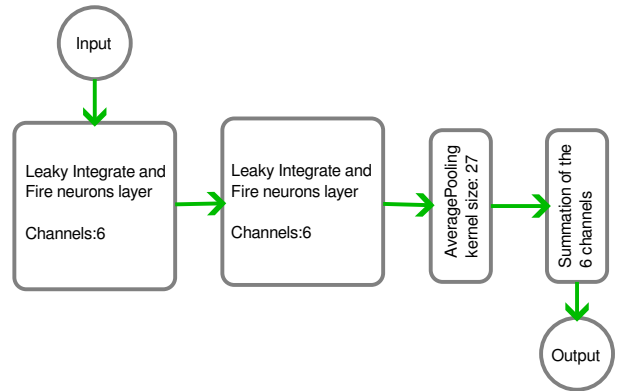


Fig. 9: Spiking neural network architecture

```
1 self.model = nn.Sequential(
2     nn.Linear(inputLength, 4096),
3     snn.Leaky(
4         beta=neuronDecayRate, spike_grad=spike_grad
5     ),
6     nn.Linear(4096, 27),
7     snn.Leaky(
8         beta=neuronDecayRate, spike_grad=spike_grad
9     ),
10    nn.AvgPool1d(27, stride=1)
11 )
```

Listing 4: SNN architecture

IV. CONCLUSIONS

As seen in the experiments SNN are a plausible alternative to a more energy heavy networks. Although the convergence went from a higher accuracy value to a lower one, overall the network proved to be more stable and, on average, had better performance.

But some questions still stand:

The Surrogate Gradient Descent is a slow method of training, its is borrowed from the standard neural networks and

adapted to SNNs. Will there be a training method that fits better for SNNs?

Another question is how energy efficient the SNNs really are in non-neuromorphic hardware?

Will neuromorphic hardware be easily available for this type of network?

V. FUTURE WORK

Implement SNNs in conjunction with another types of layers like a convolutional layer and apply some prepossessing to data using thecniques like filter banks and wavelets. Some signal processing boards and/or Field-Programmable Gate Arrays (FPGA) may be used to test the real power usage and efficiency of SNNs.

REFERENCES

- [1] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023.
- [2] W. Gerstner, W.M. Kistler, R. Naud, and L. Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.
- [3] Dan Goodman, Tomas Fiers, Richard Gao, Marcus Ghosh, and Nicolas Perez. Spiking Neural Network Models in Neuroscience - Cosyne Tutorial 2022, September 2022.
- [4] Ilenna Simone Jones and Konrad Paul Kording. Can single neurons solve mnist? the computational power of biological dendritic trees, 2020.
- [5] Nikola K Kasabov. *Time-space, spiking neural networks and brain-inspired artificial intelligence*. Springer, 2019.

VI. APPENDIX

A. 10-20 system and the brain's areas

TABLE I: Brain tasks and its corresponding regions. See Figure ?? for more information.

Region	Channels	Tasks
Frontal lobe	Fp1, Fp2, Fpz, Pz, F3, F7, F4, F8	Memory, concentration, emotions.
Parietal lobe	P3, P4, Pz	Problem Solving, attention, grammar, sense of touch.
Temporal lobe	T3, T5, T4, T6	Memory, recognition of faces, hearing, word and social clues.
Occipital lobe	O1, O2, Oz	Reading, vision.
Cerebellum		Motor control, balance.
Sensorimotor Cortex	C3, C4, Cz	Attention, mental processing, fine motor control, sensory integration.

B. Another interpretation of LIF

Starting with the Equation 7 and using Forward Euler Method to solve the LIF model:

$$\tau \cdot \frac{dV_{mem}(t)}{dt} = R \cdot I_{in}(t) - V_{mem}(t) \quad (10)$$

Solving the derivative in Equation 11 results in the membrane potential for any time $t + \Delta t$ in future:

$$\begin{aligned} \tau \cdot \frac{V_{mem}(t + \Delta t) - V_{mem}(t)}{\Delta t} &= R \cdot I_{in}(t) - V_{mem}(t) = \\ V_{mem}(t + \Delta t) - V_{mem}(t) &= \frac{\Delta t}{\tau} \cdot (R \cdot I_{in}(t) - V_{mem}(t)) = \\ \boxed{V_{mem}(t + \Delta t) &= V_{mem}(t) + \frac{\Delta t}{\tau} \cdot (R \cdot I_{in}(t) - V_{mem}(t))} \end{aligned} \quad (11)$$

C. Repositories and files

All source codes are available at:

<https://github.com/ensismoebius/deepLearnning>

The simulations and neural networks are available at:

<https://github.com/ensismoebius/deepLearnning/tree/main/simulations/article/neuralNetworks>

If you are interested in Jupyter notebooks visit:

<https://github.com/ensismoebius/deepLearnning/tree/main/simulations/article/jupyterNotebooks>

There is a nice simulator with a graphical user interface at:

<https://github.com/ensismoebius/deepLearnning/tree/main/simulations/presentation>

Browsing the location below you will have access to all latex codes used to make the documentation:

<https://github.com/ensismoebius/deepLearnning/tree/main/documentation>

Then if you want to see the log of the neural networks training access:

<https://github.com/ensismoebius/deepLearnning/tree/main/documentation/article/tables>

That's all folks!!